MSc BDA BridgeCourse Unix

UNIX COMMANDS

• When you first log into a unix system, you are presented with something “something” is called a **prompt.**

As its name would suggest, it is prompting you to enter a command.

• Every unix command is a sequence of **letters, numbers** and **characters.** But there are no spaces.

• Unix is also case-sensitive. This means that ***cat*** and ***Cat*** are different commands.

• The prompt is displayed by a special program called the **shell**.

• Shells accept commands, and run those commands.

They can also be programmed in their own language. These programs are called “**shell scripts**”.

Types of shells in unix

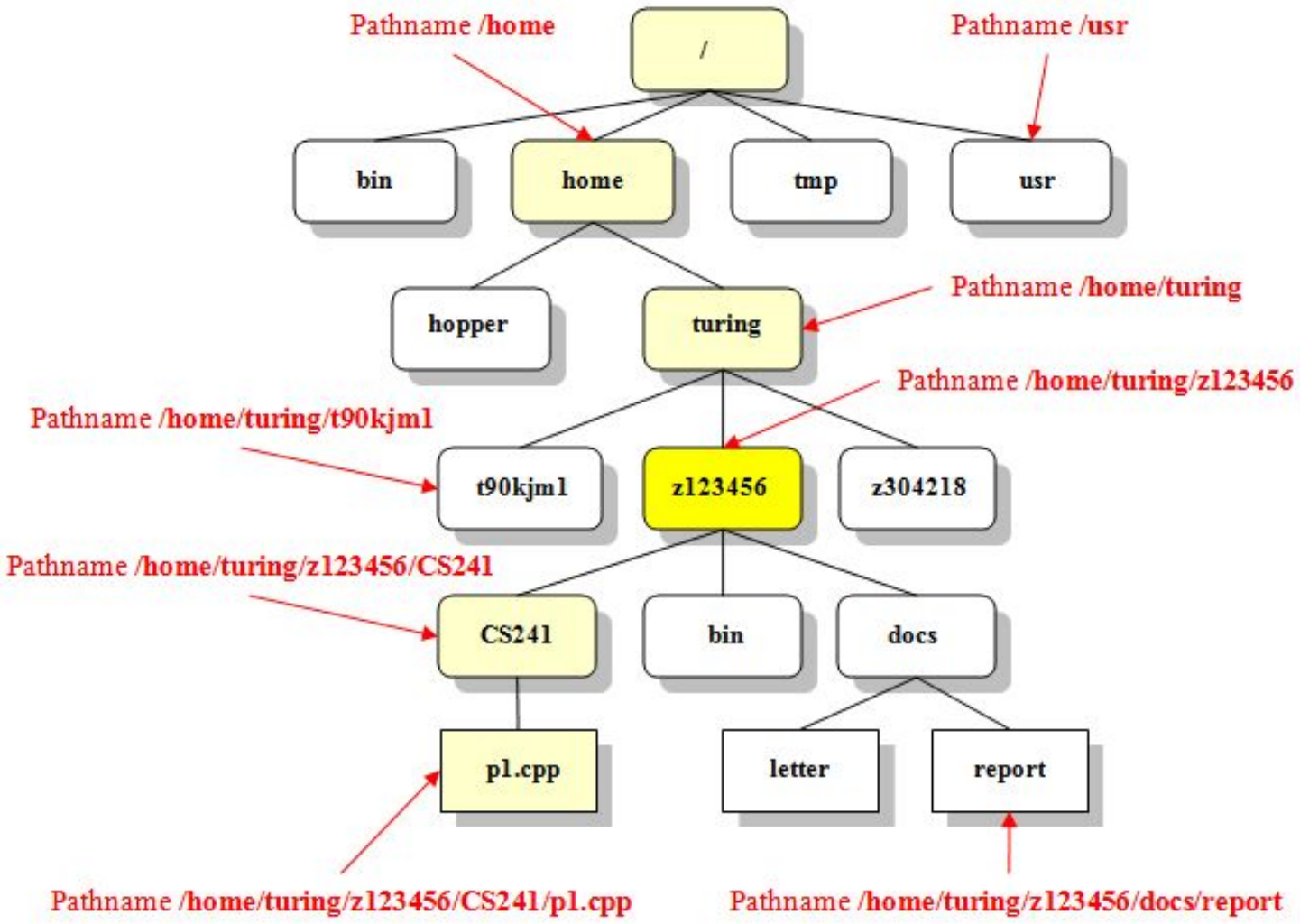
- In UNIX there are two major types of shells:
- The Bourne shell. If you are using a Bourne-type shell, the default prompt is the $ character.
- The C shell. If you are using a C-type shell, the default prompt is the % character.
- There are again various subcategories for Bourne Shell which are listed as follows:
- Bourne shell (sh)
- Korn shell (ksh)
- Bourne Again shell (bash)
- POSIX shell (sh)

The Unix File System

- The Unix file system is organized around a single structure of directories, where each directory can contain more directories (often called subdirectories) and/or files. The entire file system, often spanning many machines and disks, can be visualized as a tree. Picture this tree as growing upside down, with the root at the top and the leaves toward the bottom. The leaves are all text and executable files, while the root, trunk, limbs, branches, and twigs are all directories.

- The file system is called the directory tree, and the directory at the base of the tree is called the root directory. Every file and directory in the file system has a unique name, called its pathname. The pathname of the root directory is /.

Directory and File structure



Absolute Path Vs Relative Path In Linux:

- **Absolute Path**: An *absolute path* is defined as specifying the location of a file or directory from the root directory(/). In other words we can say *absolute path* is a complete path from start of actual filesystem from / directory.

  Relative Path: *Relative path* is defined as path related to the present working directory(pwd).

Example(I am in /home/user1 and i want to change directory to Documents)

- $ pwd
  /home/user1
  $cd Documents/ (using relative path)
  $pwd
  /home/user1/Documents

  or

  $ pwd
  /home/user1
  $cd /home/user1/Documents/ (using absolute path)
  $ pwd
  /home/user1/Documents

Login as root user.

- Username:root
- Password:system

1. mkdir

- Short for "make directory", **mkdir** is used to create directories on a file system.
-
- **--help** Display a help message, and exit.
- **-p, --parents** Create parent directories as necessary. When this option is used, no error is reported if a specified *DIRECTORY* already exists.

- Syntax: mkdir [options] dir

Q.Create a directory tree dir1/dir2/dir3 in one command.

```
[root@localhost Desktop]# mkdir --help
Usage: mkdir [OPTION]... DIRECTORY...
Create the DIRECTORY(ies), if they do not already exist.

Mandatory arguments to long options are mandatory for short options too.
  -m, --mode=MODE   set file mode (as in chmod), not a=rwx - umask
  -p, --parents     no error if existing, make parent directories as needed
  -v, --verbose     print a message for each created directory
  -Z, --context=CTX  set the SELinux security context of each created
                      directory to CTX
      --help     display this help and exit
      --version  output version information and exit

Report mkdir bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'mkdir invocation'
[root@localhost Desktop]# mkdir -p dir1/dir2/dir3
[root@localhost Desktop]#
```

2.cd

- The **cd** command, which stands for "change directory", changes the shell's current working directory.

- **Representing The Root Directory**
- The root directory itself is represented by a single slash ("**/**").
- To change into the root directory, making it our working directory, we would use the command:
- **cd /**

Representing The Working Directory

The current directory, regardless of which directory it is, is represented by a single dot ("**.**").
So, running this command:

```
cd .
```

...would change us into the current directory. In other words, it would do nothing, but it would do it successfully.

Representing The Parent Directory

The parent directory of the current directory — in other words, the directory one level up
from the current directory, which contains the directory we're in now — is represented by two dots ("**..**").
So, If we were in the directory **/home/username/documents**, and we executed the command:

```
cd ..
```

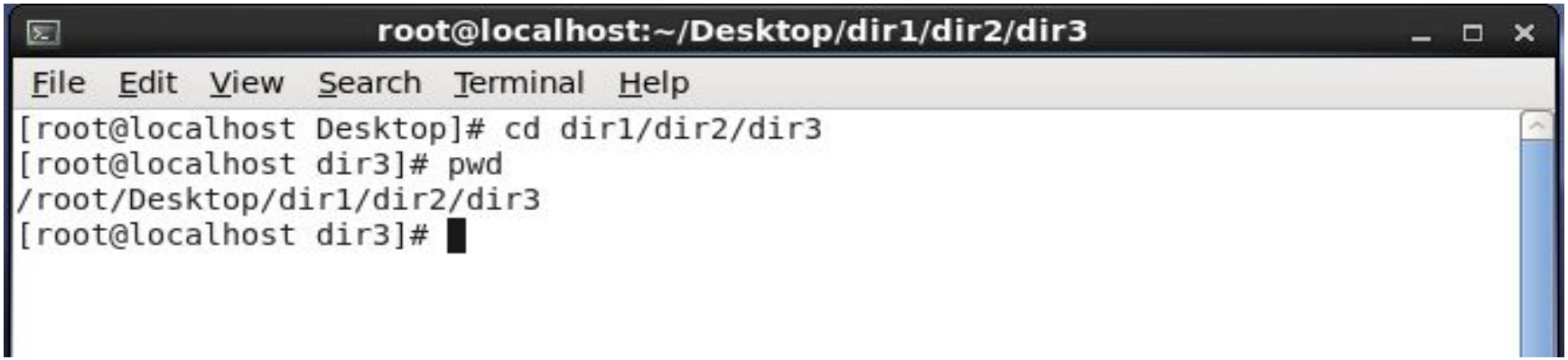
...we would be placed in the directory **/home/username**.

- Q.Change to the following directory tree (dir1/dir2/dir3) in one command.

```
[root@localhost Desktop]# cd dir1/dir2/dir3
[root@localhost dir3]# cd /
[root@localhost /]# cd .
```

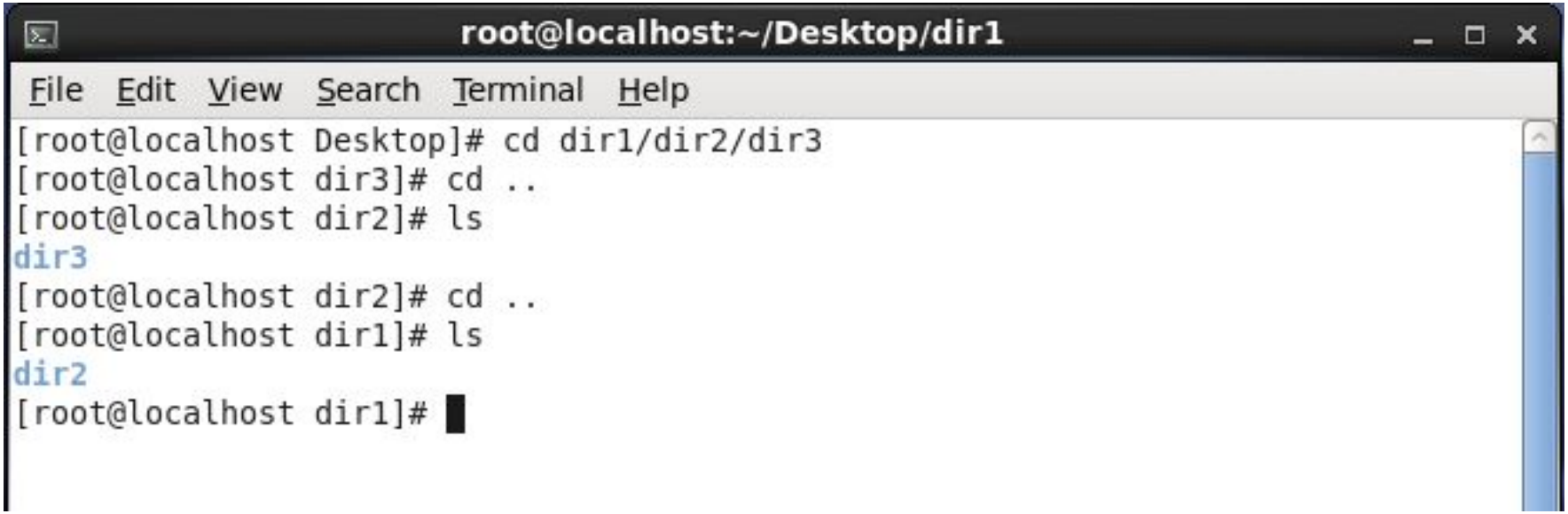
3.pwd

- **About pwd**
- Print the name of the working directory.
- **Description**
- **pwd** prints the full pathname of the current working directory.



4. ls

- **About ls**
- Lists the contents of a directory.
- **5.1)ls**



- **5.2)ls –l** (list with long format - show permissions)

5)touch

- Q. Create five empty files with the name of f1,f2,f3 ,f4 and f5.

```
[root@localhost dir1]# touch f1 f2 f3 f4 f5
[root@localhost dir1]#
```

5.3)ls –x (Multicolumnar output)

5.4) ls –t (Sorts filenames by last modification time)

5.5) ls –r (Sorts filenames in reverse order)

```
[root@localhost dir1]# ls -x
dir2  f1  f2  f3  f4  f5
[root@localhost dir1]# ls
dir2  f1  f2  f3  f4  f5
[root@localhost dir1]# ls -l
total 4
drwxr-xr-x. 3 root root 4096 Aug  8 01:10 dir2
-rw-r--r--. 1 root root    0 Aug  8 02:15 f1
-rw-r--r--. 1 root root    0 Aug  8 02:15 f2
-rw-r--r--. 1 root root    0 Aug  8 02:15 f3
-rw-r--r--. 1 root root    0 Aug  8 02:15 f4
-rw-r--r--. 1 root root    0 Aug  8 02:15 f5
[root@localhost dir1]# ls -t
f1  f2  f3  f4  f5  dir2
[root@localhost dir1]# ls -r
f5  f4  f3  f2  f1  dir2
```

6)cat

- The command is used to display the contents of a small file on the terminal.

Q.6.1)*Display Contents of File*

Q.6.2)Create a File with Cat Command

Q.6.3)View Contents of Multiple Files in terminal

```
[root@localhost dir1]# cat>f2
This is file 2
[root@localhost dir1]# cat f2
This is file 2
[root@localhost dir1]# cat>f1
This is file 1
[root@localhost dir1]# cat f1 f2
This is file 1
This is file 2
[root@localhost dir1]#
```

cat file.txt > newfile.txt

- Read the contents of **file.txt** and write them to **newfile.txt**, overwriting anything **newfile.txt** previously contained. If **newfile.txt** does not exist, it will be created.

cat file.txt >> another-file.txt

- Read the contents of **file.txt** and append them to the end of **another-file.txt**.
  If **another-file.txt** does not exist, it will be created.
-

```
[root@localhost dir1]# cat f2>>f4
[root@localhost dir1]#
```

cat –n filename

- **-n**, **--number** Number all output lines

```
[root@localhost dir1]# cat -n f4
     1  This is file 4
     2  This is file 2
     3  This is file 2
```

7) less command

Note: Take file f3 enter contents into file.

less command

"**less**" **command** is used to view files instead of opening the file.

* **Less** is a program similar to more but which allows backward movement in the file as well as forward movement

To quit: :wq

less syntax

less [*file* ...]

8) more command

About more

Displays text, one screen at a time.

Description

more is a filter for paging through text one screen at a time. It does not provide as many options or enhancements as less, but is nevertheless quite useful and simple to use.

more syntax

more [*file* ...]

9)rmdir

- The **rmdir Command**. The **rmdir command** is used to remove empty directories in **Linux** and other Unix-like operating systems.

- Q. Remove the empty directory dir3.

10)tac

Concatenate and print files in reverse.

Description

tac (which is "**cat**" backwards) concatenates each **FILE** to standard output just like the **cat** command, but in reverse: line-by-line, printing the last line first. This is useful (for instance) for examining a chronological log file in which the last line of the file contains the most recent information.

tac syntax

tac [*OPTION*] ... [*FILE*]

```
[root@localhost dir2]# cd ..
[root@localhost dir1]# tac f4
This is file 2
This is file 2
This is file 4
[root@localhost dir1]#
```

11) file

The **file** command is used to determine a file's type.

file syntax

file [*filename* ...]

- Q.11.1)Find the type of all files
- Q.11.2)Find the type of f3

```
[root@localhost dir1]# file *
dir2: directory
f1:   ASCII text
f2:   ASCII text
f3:   ASCII English text, with very long lines
f4:   ASCII text
f4~:  ASCII text
f5:   empty
[root@localhost dir1]# file f3
f3: ASCII English text, with very long lines
[root@localhost dir1]#
```

12) cp

cp command in Linux/Unix

cp is a Linux shell command to *copy* files and directories.

cp command syntax

Copy from *source* to *dest*

```
$ cp source dest
```

cp file1.txt newdir

Copies the **file1.txt** in the working directory to the **newdir** subdirectory.

13) rm

- The **rm** command removes (deletes) files or directories.

- **rm syntax**

- rm [option] filename

rm examples

rm myfile.txt

Remove the file **myfile.txt**. If the file is write-protected, you will be prompted to confirm that you really want to delete it.

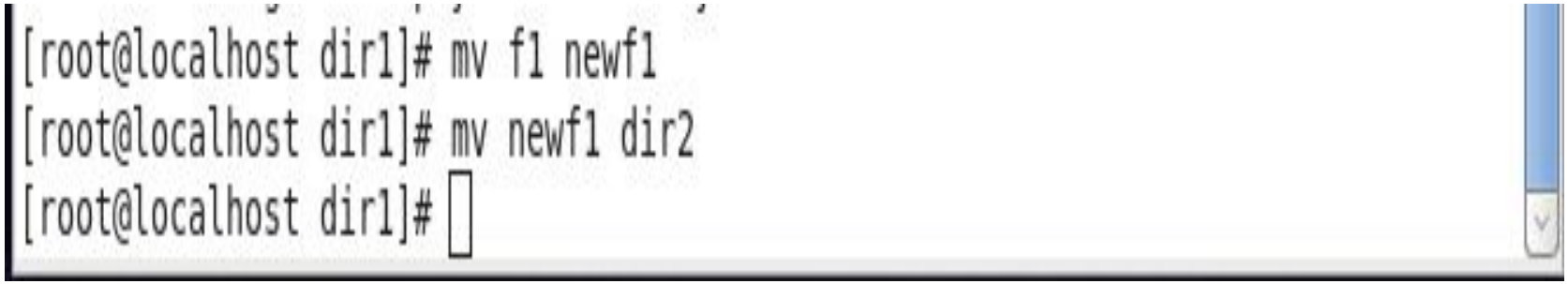
14) mv

- The **mv** command is used to move or rename files.

- **Description**
- **mv** renames file *SOURCE* to *DEST*, or moves the *SOURCE* file (or files) to *DIRECTORY*.

Q. 14.1)Rename the file f1 with newf1.

Q.14.2)Move the file newf1 to dir2



15)chmod

Change Mode (chmod)

syntax : chmod [options] mode filename

mode: read(r) , write(w), execute(x)

2 ways to give modes

1)OCTAL :

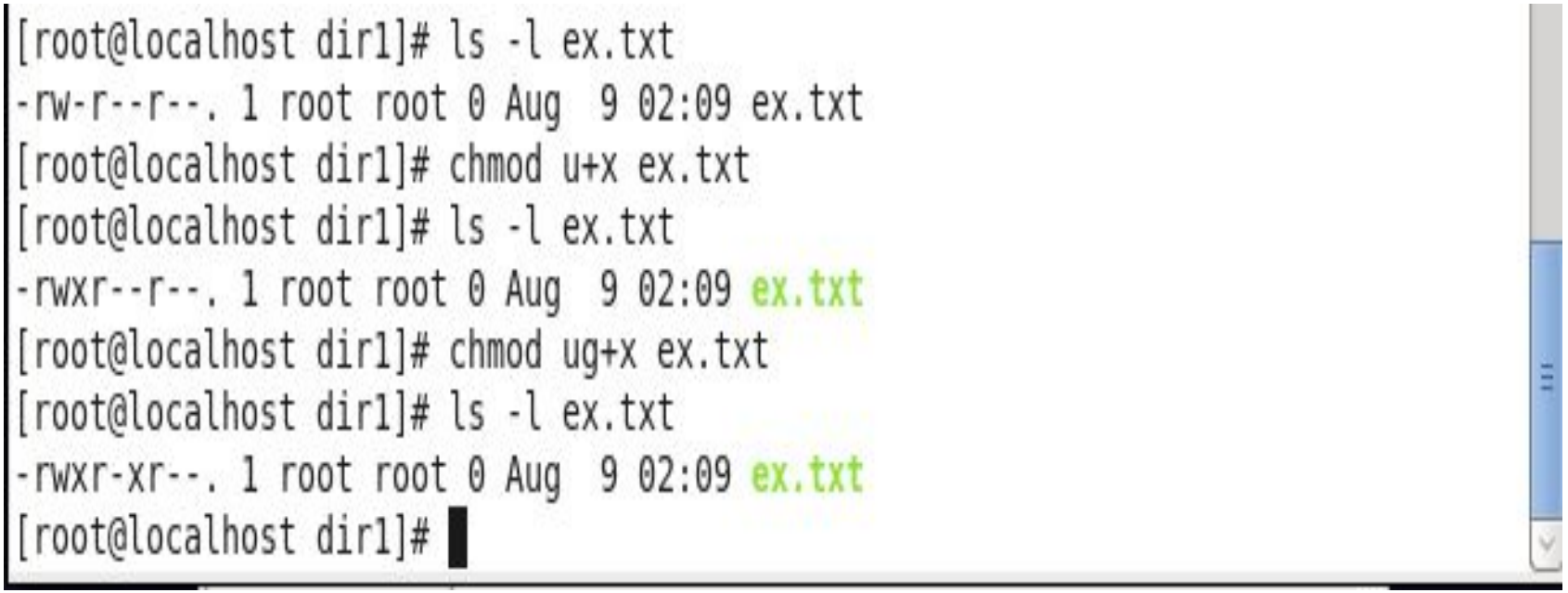
	owner			group			others		
	r	w	x	r	w	x	r	w	x
	(4)	(2)	(1)	(4)	(2)	(1)	(4)	(2)	(1)

eg: chmod -R 774 filename

```
2)Symbolic:

        syntax: chmod references operator modes file

references :    u: owner of the file
                g: users who are member of file's group
                o: users who are neither u nor g
                a: all ugo

operator:       +: adds mode to specified classes
                -: remove mode
                =: exact mode to specified classes

mode            r:read
                w:write
                x:execute
```

Eg: chmod u+x filename

- Create a file in dir1 as ex.txt



16)head

The *head* command reads the first few lines of any text given to it as an input and writes them to *standard output* (which, by default, is the display screen).

head's basic syntax is:

head [options] [file(s)]

head examples

Display the first fifteen lines of **myfile.txt**.

head -15 myfile.txt

17)tail

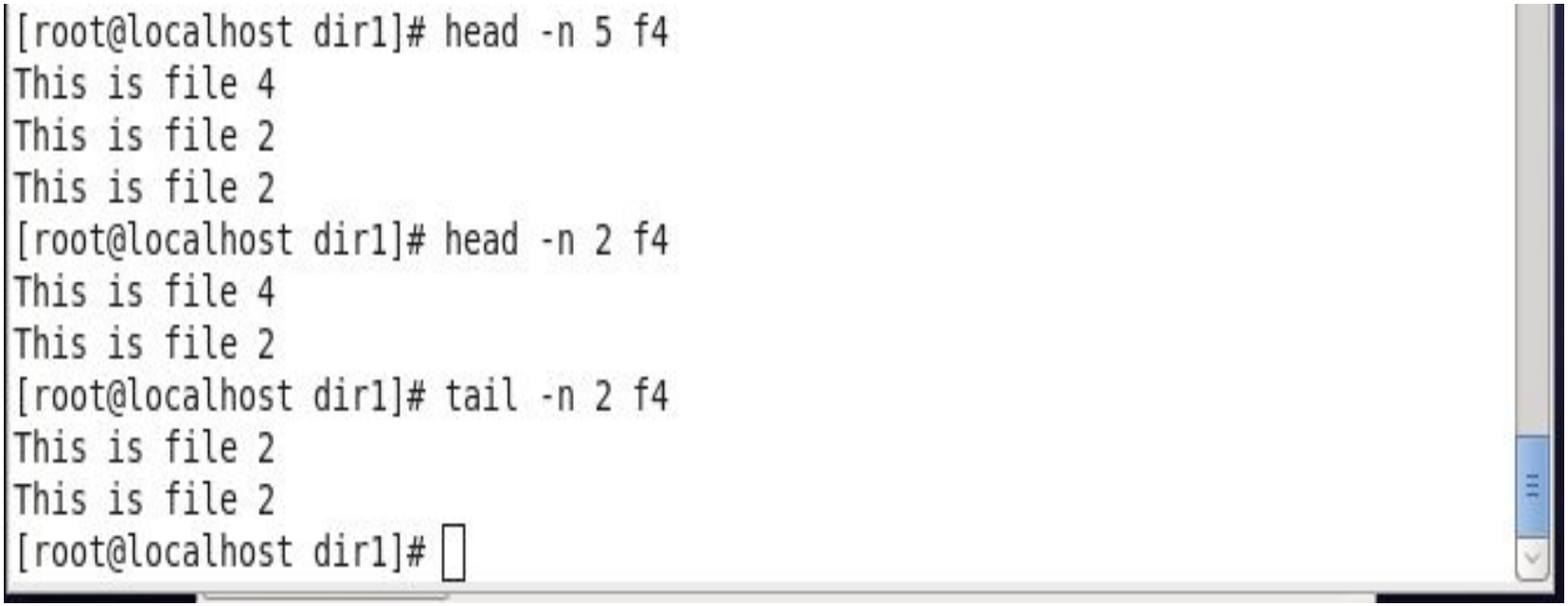
- **tail** outputs the last part, or "tail", of files.

Description

- **tail** prints the last 10 lines of each FILE to standard output. With more than one FILE, it precedes each set of output with a header giving the file name. If no FILE is specified, or if FILE is specified as a dash ("**-**"), **tail** reads from standard input.

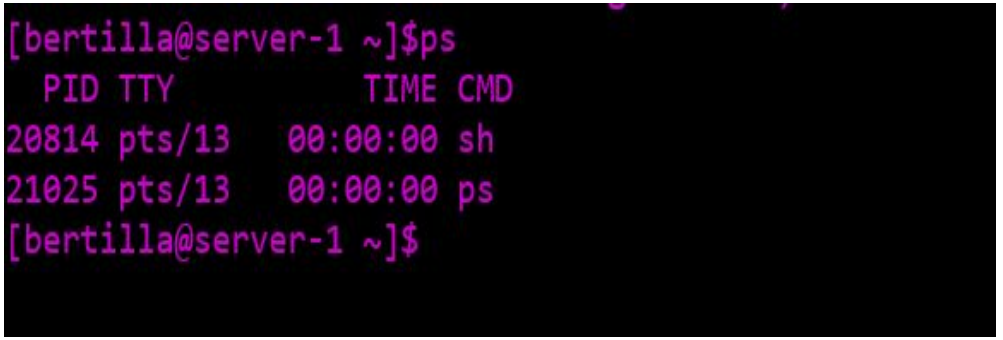
Q.17.1)Display the first 2 lines of file f4.

Q.17.2)Display the last 2 lines of file f4.



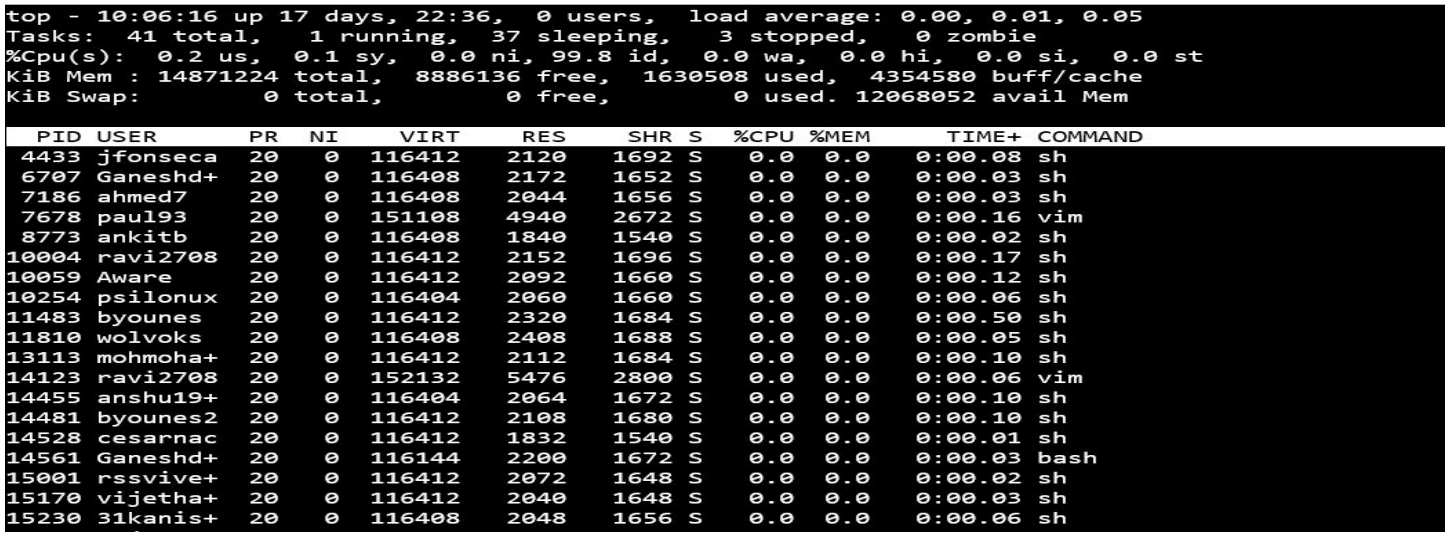
Ps command

- The **ps** (i.e., **process status**) **command** is used to provide information about the currently running processes, including their process identification numbers (PIDs). A process, also referred to as a task, is an executing (i.e., running) instance of a program. Every process is assigned a unique PID by the system



top command

- top **command displays processor activity** of your Linux box and also displays tasks managed by kernel in real-time. It'll show processor and memory are being used and other information like running processes.

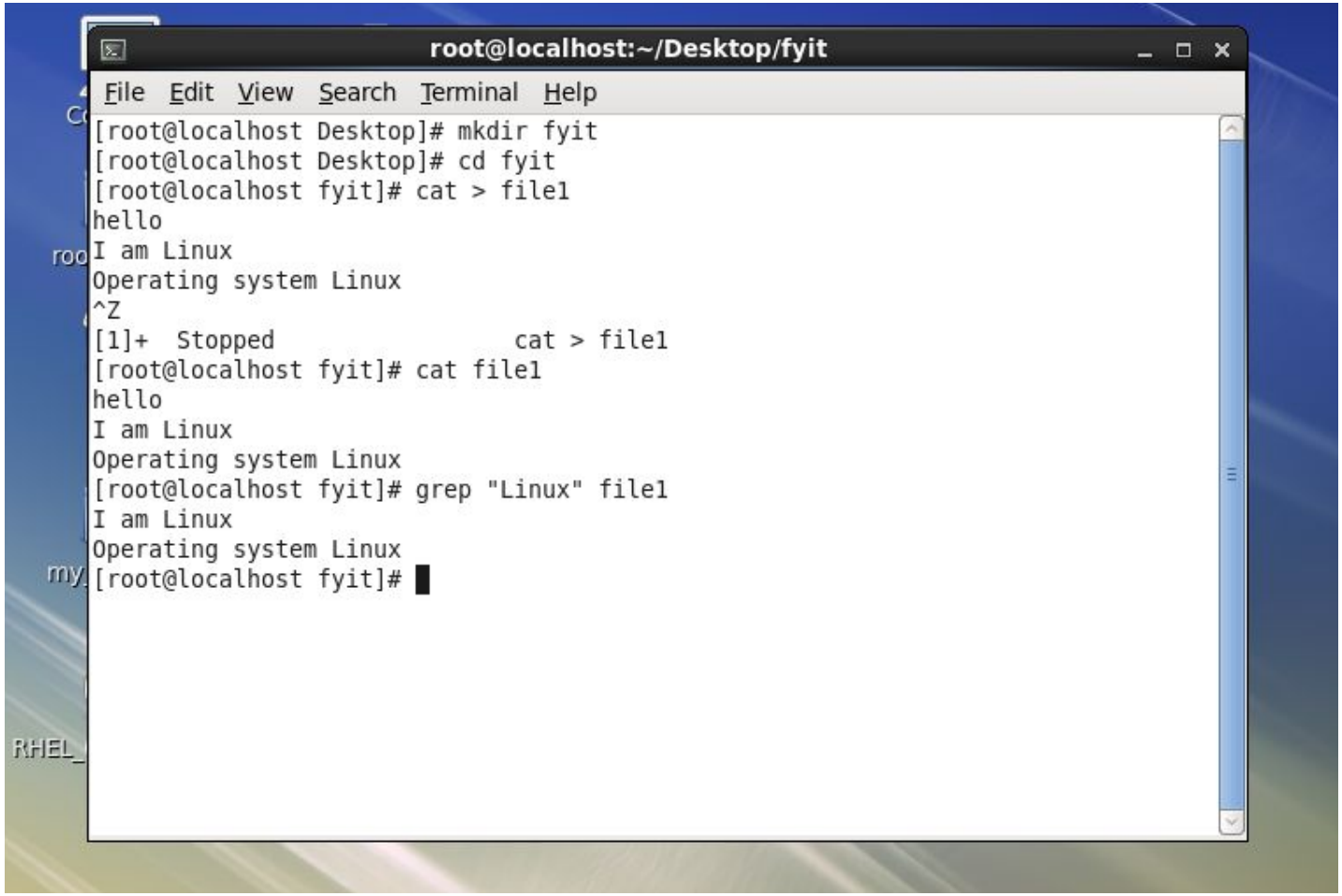


```
top - 10:06:16 up 17 days, 22:36,  0 users,  load average: 0.00, 0.01, 0.05
Tasks:  41 total,   1 running,  37 sleeping,   3 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.1 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 14871224 total,  8886136 free,  1630508 used,  4354580 buff/cache
KiB Swap:        0 total,        0 free,        0 used. 12068052 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 4433 jfonseca  20   0  116412   2120   1692 S   0.0  0.0   0:00.08 sh
 6707 Ganeshd+  20   0  116408   2172   1652 S   0.0  0.0   0:00.03 sh
 7186 ahmed7    20   0  116408   2044   1656 S   0.0  0.0   0:00.03 sh
 7678 paul93    20   0  151108   4940   2672 S   0.0  0.0   0:00.16 vim
 8773 ankitb    20   0  116408   1840   1540 S   0.0  0.0   0:00.02 sh
10004 ravi2708  20   0  116412   2152   1696 S   0.0  0.0   0:00.17 sh
10059 Aware     20   0  116412   2092   1660 S   0.0  0.0   0:00.12 sh
10254 psilonux  20   0  116404   2060   1660 S   0.0  0.0   0:00.06 sh
11483 byounes   20   0  116412   2320   1684 S   0.0  0.0   0:00.50 sh
11810 wolvoks   20   0  116408   2408   1688 S   0.0  0.0   0:00.05 sh
13113 mohmoha+  20   0  116412   2112   1684 S   0.0  0.0   0:00.10 sh
14123 ravi2708  20   0  152132   5476   2800 S   0.0  0.0   0:00.06 vim
14455 anshu19+  20   0  116404   2064   1672 S   0.0  0.0   0:00.10 sh
14481 byounes2  20   0  116412   2108   1680 S   0.0  0.0   0:00.10 sh
14528 cesarnac  20   0  116412   1832   1540 S   0.0  0.0   0:00.01 sh
14561 Ganeshd+  20   0  116144   2200   1672 S   0.0  0.0   0:00.03 bash
15001 rssvive+  20   0  116412   2072   1648 S   0.0  0.0   0:00.02 sh
15170 vijetha+  20   0  116412   2040   1648 S   0.0  0.0   0:00.03 sh
15230 31kanis+  20   0  116408   2048   1656 S   0.0  0.0   0:00.06 sh
```

grep command

- grep, which stands for "global regular expression print," processes text line by line and prints any lines which match a specified pattern.
- grep syntax
- grep [OPTIONS] PATTERN [FILE...]
- Case insensitive search

- The -i option enables to search for a string case insensitively in the give file. It matches the words like "UNIX", "Unix", "unix".
- Example: grep -i "UNix" file.txt

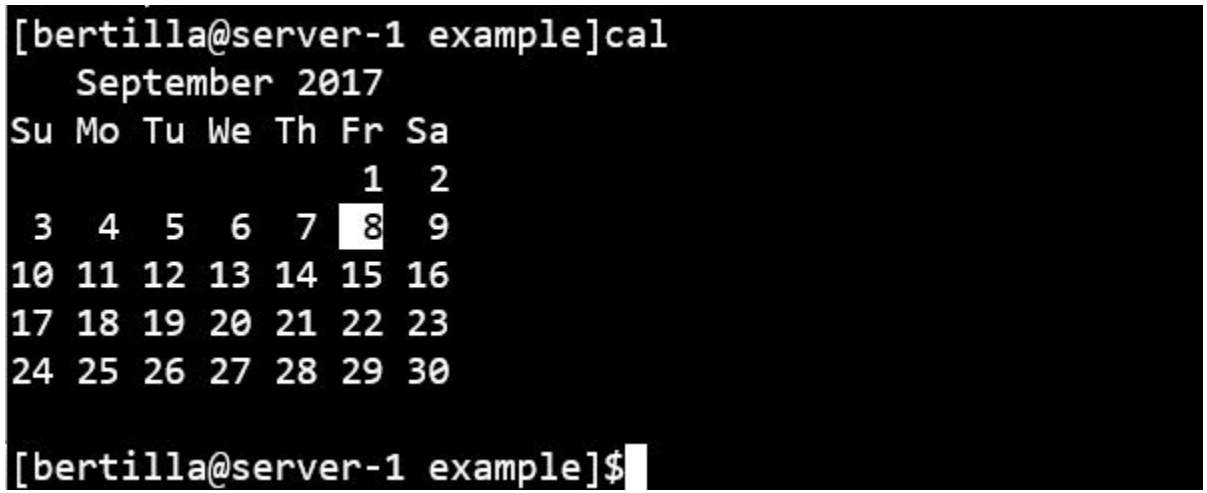

root@localhost:~/Desktop/fyit
File Edit View Search Terminal Help
[root@localhost Desktop]# mkdir fyit
[root@localhost Desktop]# cd fyit
[root@localhost fyit]# cat > file1
hello
I am Linux
Operating system Linux
^Z
[1]+ Stopped cat > file1
[root@localhost fyit]# cat file1
hello
I am Linux
Operating system Linux
[root@localhost fyit]# grep "Linux" file1
I am Linux
Operating system Linux
[root@localhost fyit]#

date command

- The **date command** displays the current **date** and time. It can also be used to display or calculate a **date** in a format you specify. The super-user (root) can use it to set the system clock.

cal command

- If you want to quickly view a calendar on terminal in **Linux**, then **cal** is the **command** line tool that you should be using. By default the **command** displays the current month in output.



whoami command

- It is a concatenation of the words "**Who am I**?" and prints the effective username of the current user when invoked.

```
[bertilla@server-1 example]$whoami
bertilla
[bertilla@server-1 example]$
```

uname command

- Print information about the current system.

```
[bertilla@server-1 ~]$uname
Linux
```

man command

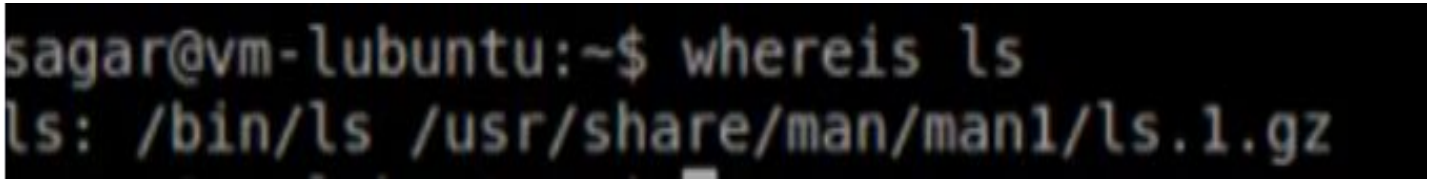
- The **man Command**. The **man command** is used to format and display the **man** pages. The **man** pages are a user manual that is by default built into most **Linux** distributions (i.e., versions) and most other Unix-like operating systems during installation

- Eg: man ls

- ls --help

whereis command

- Locates the binary, source, and manual page files for a command.
- Example:



Piping in Unix or Linux

- A pipe is a form of redirection (transfer of standard output to some other destination) that is used in Linux and other Unix-like operating systems to send the output of one command/program/process to another command/program/process for further processing. The Unix/Linux systems allow stdout of a command to be connected to stdin of another command. You can make it do so by using the pipe character **'|'**.
- Pipe is used to combine two or more commands, and in this, the output of one command acts as input to another command, and this command's output may act as input to the next command and so on. It can also be visualized as a temporary connection between two or more commands/ programs/ processes. The command line programs that do the further processing are referred to as filters.
- This direct connection between commands/ programs/ processes allows them to operate simultaneously and permits data to be transferred between them continuously rather than having to pass it through temporary text files or through the display screen.
  Pipes are unidirectional **i.e data flows from left to right through the pipeline.**

Syntax :

```
command_1 | command_2 | command_3 | .... | command_N
```

Examples

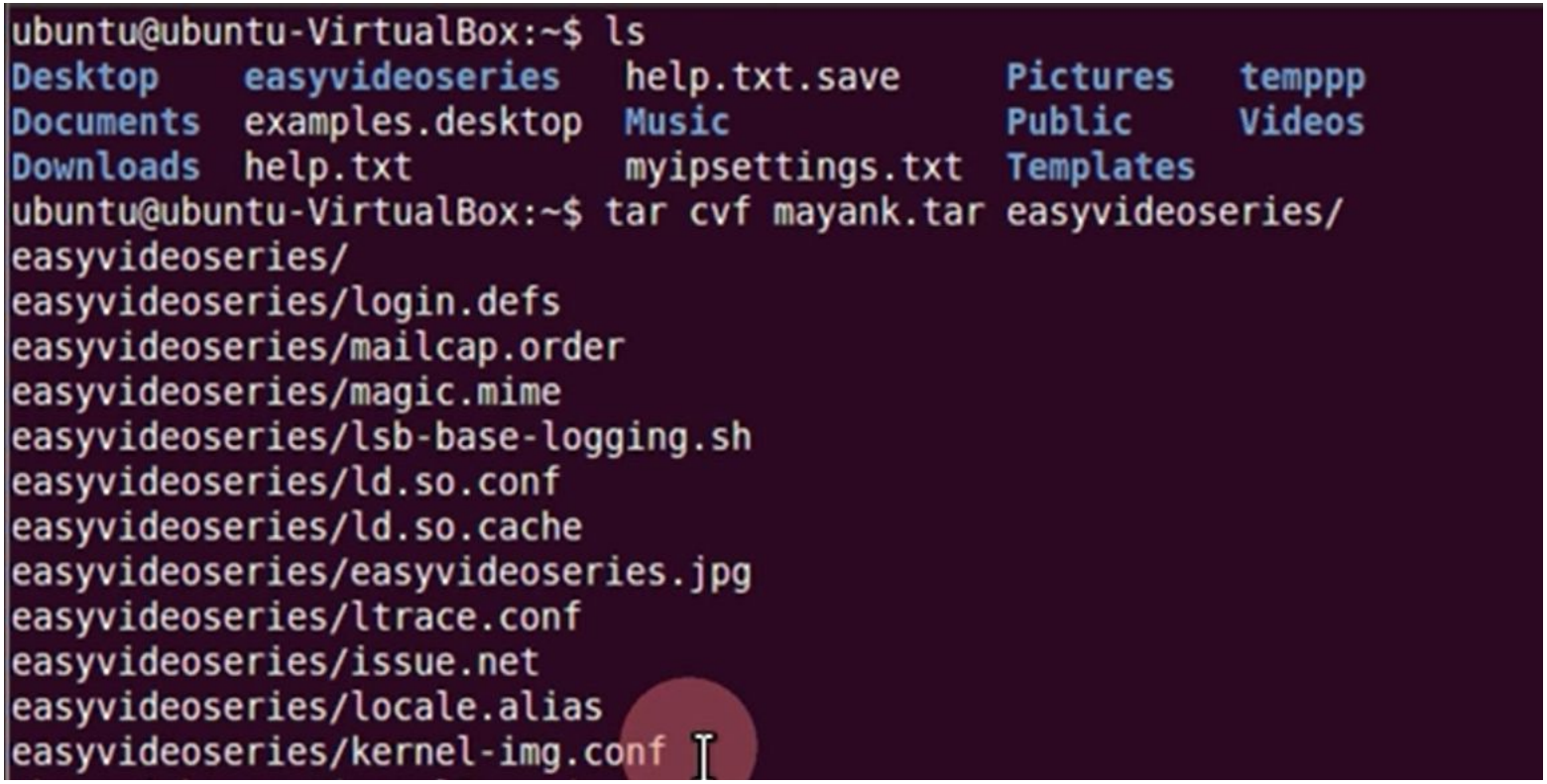
. Use head and tail to print lines in a particular range in a file.

```
$ cat sample2.txt | head -7 | tail -5
```

```
localhost:~# cat test | grep "one"
```

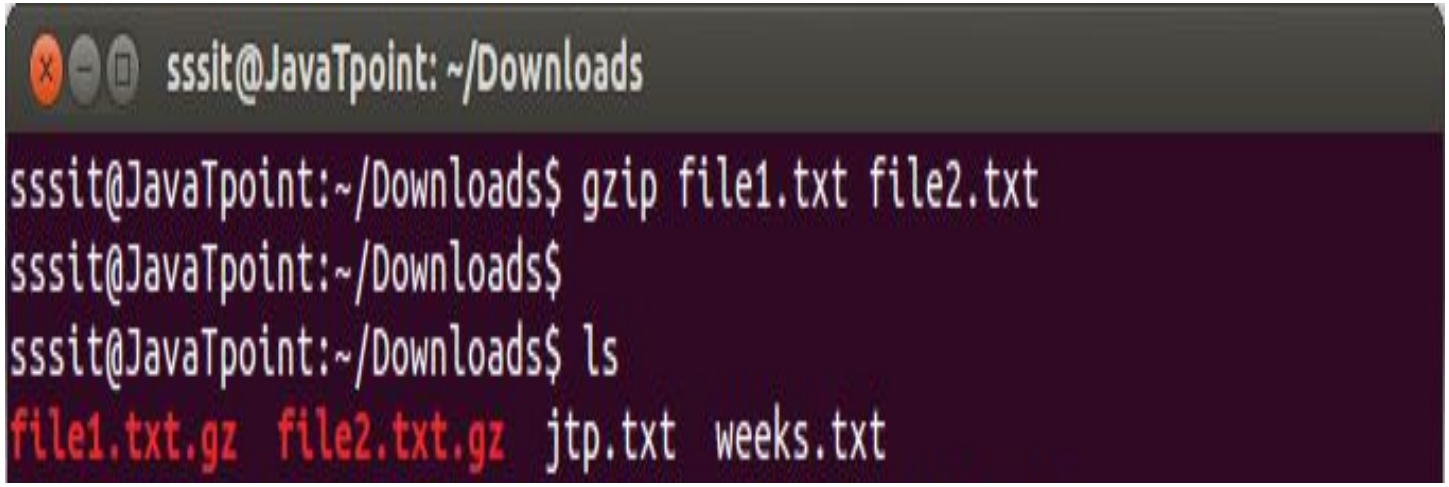
tar command:

- The Linux “tar” stands for tape archive, which is used by large number of Linux/Unix system administrators to deal with tape drives backup. The tar command used to rip a collection of files and directories into highly compressed archive file commonly called tarball or tar, gzip and bzip in Linux. The tar is most widely used command to create compressed archive files and that can be moved easily from one disk to anther disk or machine to machine.
- c – Creates a new .tar archive file.
- v – Verbosely show the .tar file progress.
- f – File name type of the archive file.


ubuntu@ubuntu-VirtualBox:~$ ls
Desktop easyvideoseries help.txt.save Pictures temppp
Documents examples.desktop Music Public Videos
Downloads help.txt myipsettings.txt Templates
ubuntu@ubuntu-VirtualBox:~$ tar cvf mayank.tar easyvideoseries/
easyvideoseries/
easyvideoseries/login.defs
easyvideoseries/mailcap.order
easyvideoseries/magic.mime
easyvideoseries/lsb-base-logging.sh
easyvideoseries/ld.so.conf
easyvideoseries/ld.so.cache
easyvideoseries/easyvideoseries.jpg
easyvideoseries/ltrace.conf
easyvideoseries/issue.net
easyvideoseries/locale.alias
easyvideoseries/kernel-img.conf

gzip command:

- The "gzip" command is a common way of compressing files within Linux
- The simplest way to compress a single file using gzip is to run the following command:
- ->gzip filename


sssit@JavaTpoint: ~/Downloads
sssit@JavaTpoint:~/Downloads$ gzip file1.txt file2.txt
sssit@JavaTpoint:~/Downloads$
sssit@JavaTpoint:~/Downloads$ ls
file1.txt.gz file2.txt.gz jtp.txt weeks.txt

Vi Editor

- **To Start vi**
- To use vi on a file, type in vi filename. If the file named filename exists, then the first page (or screen) of the file will be displayed; if the file does not exist, then an empty file and screen are created into which you may enter text.

*	vi filename	*edit filename starting at line 1*

To Exit vi

- Usually the new or modified file is saved when you leave vi. However, it is also possible to quit vi without saving the file.

*	:x<Return>	*quit vi, writing out modified file to file named in original invocation*
	:wq<Return>	*quit vi, writing out modified file to file named in original invocation*
	:q<Return>	*quit (or exit) vi*

Moving the Cursor

- Unlike many of the PC and MacIntosh editors, **the mouse does not move the cursor** within the vi editor screen (or window). You must use the the key commands listed below. On some UNIX platforms, the arrow keys may be used as well; however, since vi was designed with the Qwerty keyboard (containing no arrow keys) in mind, the arrow keys sometimes produce strange effects in vi and should be avoided.
- If you go back and forth between a PC environment and a UNIX environment, you may find that this dissimilarity in methods for cursor movement is the most frustrating difference between the two.
- In the table below, the symbol ^ before a letter means that the <Ctrl> key should be held down while the letter key is pressed.

Moving the Cursor

*	j *or* <Return> [*or* down-arrow]	*move cursor down one line*
*	k [*or* up-arrow]	*move cursor up one line*
*	h *or* <Backspace> [*or* left-arrow]	*move cursor left one character*
*	l *or* <Space> [*or* right-arrow]	*move cursor right one character*
*	0 (zero)	*move cursor to start of current line (the one with the cursor)*
*	$	*move cursor to end of current line*
	w	*move cursor to beginning of next word*
	b	*move cursor back to beginning of preceding word*

Write a Hello World C Program

```
$ vim helloworld.c

/* Hello World C Program */

#include<stdio.h>

main()
{
   printf("Hello World!");

}
```

Compile the helloworld.c Program

- Compile the helloworld.c using cc command as shown below. This will create the a.out file.

- $ cc helloworld.c

Execute the C Program (a.out)

- You can either execute the a.out to see the output (or) rename it to some other meaningful name and execute it as shown below.

- $ ./a.out
- Hello World!

Shell Programs

- Write script to print nos as 5,4,3,2,1 using while loop

```
#!/bin/bash
#
# Linux Shell Scripting Tutorial 1.05r3, Summer-2002
#
# Written by Vivek G. Gite <vivek@nixcraft.com>
#
# Latest version can be found at http://www.nixcraft.com/
#
# Q3
# Algo:
#       1) START: set value of i to 5 (since we want to start from 5, if you
#          want to start from other value put that value)
#       2) Start While Loop
#       3) Chechk, Is value of i is zero, If yes goto step 5 else
#          continue with next step
#       4) print i, decement i by 1 (i.e. i=i-1 to goto zero) and
#          goto step 3
#       5) END
#
i=5
while test $i != 0
do
        echo "$i
"
        i=`expr $i - 1`
done
```

Executing a shell program